

ATPCS

为使单独编译的C语言程序和汇编程序之间能够相互调用,必须为程序间的调用规定一定规则. ATPCS就是ARM程序和Thumb程序中子程序调用的基本规则

ATPCS规定了一些子程序间调用的基本规则,包括子程序调用过程中寄存器的使用规则,数据栈的使用规则,参数的传递规则.

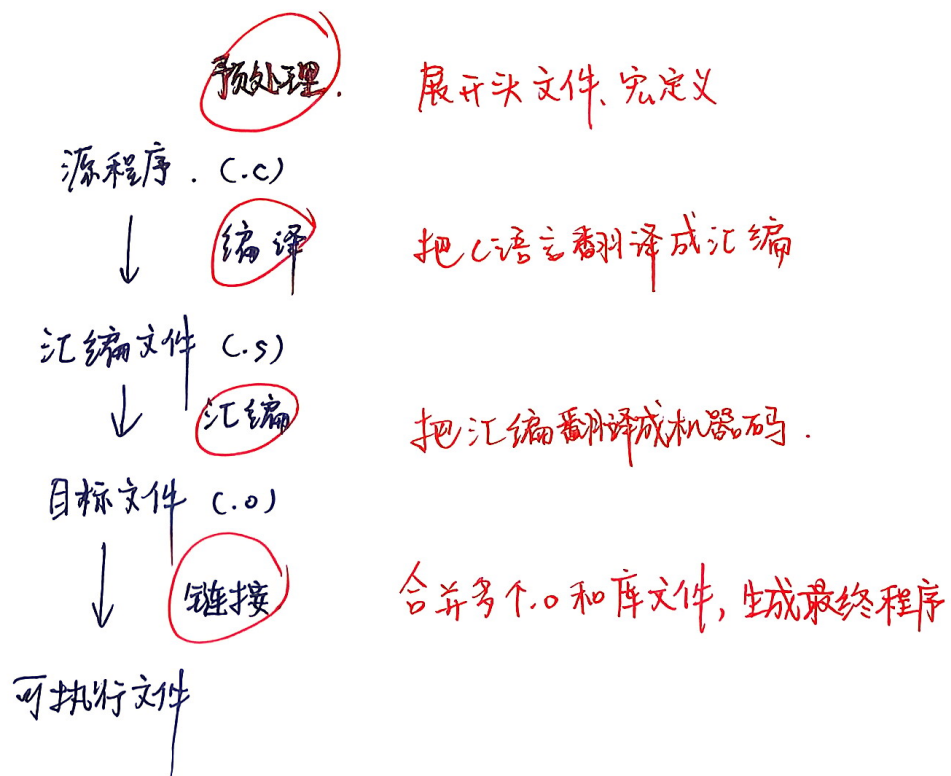
为适应一些特定的需要,对这些基本调用规则进行一些修改得到几种不同的子程序调用规则,这些特定的调用规则包括:

- ❑ 支持数据栈限制检查的 ATPCS。
- ❑ 支持只读段位置无关(ROPI)的 ATPCS。
- ❑ 支持可读写段位置无关(RWPI)的 ATPCS。
- ❑ 支持 ARM 程序和 Thumb 程序混合使用的 ATPCS。
- ❑ 处理浮点运算的 ATPCS。
- ❑ 有调用关系的所有子程序必须遵守同一种 ATPCS。编译器或者汇编器在 ELF 格式的目标文件中设置相应的属性，标识用户选定的 ATPCS 类型。
- ❑ 对应于不同类型的 ATPCS 规则，有相应的 c 语言库，连接器根据用户指定的 ATPCS 类型连接相应的 c 语言库。
- ❑ 使用 C 语言编译器编译的 C 语言子程序满足用户指定的 ATPCS 类型。而对于汇编语言程序来说，完全要依赖用户来保证各子程序满足选定的 ATPCS 类型。具体来说，汇编语言子程序必须满足下面 3 个条件：
 - 1) 在子程序编写时必须遵守相应的 ATPCS 规则。
 - 2) 数据栈的使用要遵守相应的 ATPCS 规则。
 - 3) 在汇编编译器中使用 ATPCS 选项。
- ❑ 基本 ATPCS 规定了在子程序调用时的一些基本规则，包括下面 3 方面的内容：
 - 1) 各寄存器的使用规则及其相应的名称。
 - 2) 数据栈的使用规则。
 - 3) 参数传递的规则。
- ❑ 相对于其他类型的 ATPCS，满足基本 ATPCS 的程序的执行速度更快，所占用的内存更少。但是它不能提供以下的支持：
 - 1) ARM 程序和 Thumb 程序相互调用。
 - 2) 数据以及代码的位置无关的支持。
 - 3) 子程序的可重入性。
 - 4) 数据栈检查的支持。
- ❑ 而派生的其他几种特定的 ATPCS 就是在基本 ATPCS 的基础上再添加其他的规则而形成的。其目的就是提供上述的功能。
- ❑ 寄存器的使用必须满足下面的规则：
 - ① 子程序间通过寄存器 R0-R3 来传递参数。这时，寄存器 R0-R3 可以记作 A1-A4。被调用的子程序在返回前无需恢复寄存器 R0-R3 的内容。
 - ② 在子程序中，使用寄存器 R4-R11 来保存局部变量。这时，寄存器 R4-R11 可以记作 V1-V8。如果在子程序中使用到了寄存器 V1-V8 中的某些寄存器，子程序进入时必须保存这些寄存器的值，在返回前必须恢复这些寄存器的值；对于子程序中没有用到的寄存器则不必进行这些操作。在 Thumb 程序中，通常只能使用寄存器 R4-R7 来保存局部变量。
 - ③ 寄存器 R12 用作子程序间 临时的 scratch 寄存器（用于保存 SP，在函数返回时使用该寄存器出栈），记作 ip。在子程序间的连接代码段中常有这种使用规则。
 - ④ 寄存器 R13 用作数据栈指针，记作 sp。在子程序中寄存器 R13 不能用作其他用途。寄存器 sp 在进入子程序时的值和退出子程序时的值必须相等。
 - ⑤ 寄存器 R14 称为连接寄存器，记作 lr。它用于保存子程序的返回地址。如果在子程序中保存了返回地址，寄存器 R14 则可以用作其他用途。
 - ⑥ 寄存器 R15 是程序计数器，记作 pc。它不能用作其他用途。

ip 子程序调用过程中
的临时寄存器

□ 下表总结了在 ATPCS 中各寄存器的使用规则及其名称。这些名称在编译器和汇编器中都是预定义的：

寄存器	别名	特殊名称	使用规则
R15	PC	pc	程序计数器
R14	LR	lr	连接寄存器，保存子程序返回地址
R13	SP	sp	数据栈指针
R12	IP	ip	子程序内部调用的 scratch 寄存器
R11	V8		ARM 状态局部变量寄存器 8
R10	V7	sl	ARM 状态局部变量寄存器 7； 在支持数据栈检查的 ATPCS 中为数据栈限制指针
R9	V6	sb	ARM 状态局部变量寄存器 6； 在支持 RWPI 的 ATPCS 中为静态基址寄存器
R8	V5		ARM 状态局部变量寄存器 5
R7	V4	wr	局部变量寄存器 4； Thumb 状态工作寄存器
R6	V3		局部变量寄存器 3
R5	V2		局部变量寄存器 2
R4	V1		局部变量寄存器 1
R3	A4		参数 / 结果 / scratch 寄存器 4
R2	A3		参数 / 结果 / scratch 寄存器 3
R1	A2		参数 / 结果 / scratch 寄存器 2
R0	A1		参数 / 结果 / scratch 寄存器 1



数据栈使用规则

栈指针通常可以指向不同的位置。当栈指针指向栈顶元素（最后一个入栈的元素）时，称为FULL栈。当栈顶指针指向与栈顶元素相邻的一个可用数据单元时，称为EMPTY栈。

数据栈的增长方向也可以不同。当数据栈向内存地址减小的方向增长时，称为DESCENDING栈。当数据栈向内存地址增加的方向增长时，称为ASCENDING栈。

综合以上2种特点可有以下4种数据栈：

FD	ED	FA	EA
"	"	"	"
Full Descending	Empty Descending	Full Ascending	Empty Descending

栈指针SP常保持在8的倍数地址上

ATPCS规定数据栈为FD类型，并且对数据栈的操作是8字节对齐的。

异常中断的处理程序可以使用被中断程序的数据栈，这时用户要保证中断的程序的数

中断发生时，中断处理程序可以继续用原来程序正在用的那个栈

据栈为中断处理预留了足够的空间

参数传递规则

根据参数个数是否固定可将子程序分为参数固定的(nonvariadic)子程序和参数个数可变的(variadic)子程序。这2种子程序的参数传递规则不同

① 参数个数可变的子程序参数传递规则

当参数不超过4个时，可使用寄存器R0-R3来传递参数；超过4个时，用数据栈来传递参数。

参数传递时，将所有参数看作是存放在连续内存字单元中的字数据。

然后，依次将各字数据传送到寄存器R0、R1、R2、R3中。

★ 如果参数多于4个，则将剩余的字数据传送到数据栈中，入栈顺序与参数顺序相反。

最后一个字数据先入栈

假如把4个寄存器R0-R3中的4个字数据写入栈中内存地址1000-1015的地方。

若用前面那个AA PUSH多寄存器指令就不用反着来。PUSH {R0, R1, R2, R3} PUSH {R0-R3}
PUSH {R3, R2, R1, R0} 都有一个效果

若一个一个PUSH就要反着来

PUSH {R3}

PUSH {R2}

PUSH {R1}

PUSH {R0}

② 参数个数固定的子程序参数传递规则

与前者不同,若系统包含浮点运算的硬件部件,浮点参数将按以下规则传递:

- 各浮点参数按顺序处理
- 为每个浮点参数分配FP寄存器
- 分配方法是,满足该浮点参数需要的且编号最小的一组连续的FP寄存器
- 第一个整数参数通过寄存器R0-R3来传递,其他参数通过数据栈传递

子程序结果返回规则

1. 结果为一个 32 位的整数时, 可以通过寄存器 R0 返回。
2. 结果为一个 64 位整数时, 可以通过寄存器 R0 和 R1 返回, 依次类推。
3. 结果为一个浮点数时, 可以通过浮点运算部件的寄存器 f0, d0 或者 s0 来返回。
4. 结果为复合型的浮点数(如复数)时, 可以通过寄存器 f0~fn 或者 d0~dn 来返回。
5. 对于位数更多的结果, 需要通过内存来传递。→ 调用者准备一块内存, 然后把这块内存的地址

ARM 程序和 Thumb 程序混合使用的 ATPCS

传给被调用函数. 被调用函数把结果写进这块内存

- 在编译和汇编时, 使用 `/interwork` 告诉编译器生成的目标代码遵守支持 ARM 程序和

Thumb 程序混合使用的 ATPCS。它用在以下场合:

告诉编译器生成的代码要支持 ARM 和 Thumb 函数互相调用。

1. 程序中存在 ARM 程序调用 Thumb 程序的情况。
2. 程序中存在 Thumb 程序调用 ARM 程序的情况。
3. 需要连接器来进行 ARM 状态和 Thumb 状态切换的情况。

- 在下述情况下, 使用选项 `/nointerwork`。

1. 程序中不包含 Thumb 程序。
2. 用户自己进行 ARM 状态和 Thumb 状态切换。

- 其中, 选项 `/nointerwork` 是默认的选项。

可以把汇编写进去, "内联汇编"

- 需要注意的是, 在同一个 C/C++ 源程序中不能同时包含 ARM 指令和 Thumb 指令, 但汇编可以。

单个文件只能是纯 ARM 或纯 Thumb

- 如果程序遵守支持 ARM 程序和 Thumb 程序混合使用的 ATPCS, 则程序中 ARM 子程序和 Thumb 子程序可以相互调用。对于 C/C++ 源程序而言, 只要在编译时指定 `apcs / interwork` 选项, 编译器生成的代码会自动遵守支持 ARM 程序和 Thumb 程序混合使用的 ATPCS。而对于汇编源程序而言, 必须保证编写的代码遵守支持 ARM 程序和 Thumb 程序混合使用的 ATPCS。

对整个项目, 可以

ARM 文件与 Thumb 文件共存, 靠 `interwork` 相互调用

在汇编语言程序中通过用户代码支持 `interwork`

ARM-Thumb Procedure Call Standard

在版本 4 中可以实现程序状态切换的指令是 `BX`。

从版本 5 开始, 下面的指令也可以实现程序状态的切换:

- `BLX`
- `LDR`, `LDM` 及 `POP`

ARM/Thumb 切换的关键在于跳转目标地址的最低位 bit0

目标地址 bit0 = 0 表示进入 ARM 状态

目标地址 bit0 = 1 表示进入 Thumb 状态

注意! 真正的指令地址还是对齐的, 最低位不是实际地址的一部分, 而是用于表示状态

如何在汇编程序中实现状态切换的汇编程序实例

从 ARM 状态切换到

Thumb 状态, 再以

Thumb 状态切换回 ARM 状态

```
ARM
ADR r0, ThumbProg+1
BX r0 → ARM 指令 跳到 ThumbProg, 程序切换到 Thumb 状态*/
.THUMB
/*THUMB 指示编译器后面的为 Thumb 指令*/
ThumbProg:
...
ADR r0, ARMProg
BX r0 → Thumb 指令 跳转到 ARMProg, 程序切换到 ARM 状态*/
/*ARM 指示编译器后面的为 ARM 指令*/
ARMProg:
MOV r4, #4 → ARM 指令 /*把立即数 4 放入 R4 中*/
```

ADR:

把某个标号的地址计算出来, 放入寄存器

标签: 地址标记. 是 ARM 程序段的入口位置, 叫 ARMProg. 前面 ADR r0, ARMProg 就是把 ARMProg 这个地址放进 R0. 然后 BX R0 切回 ARM 状态, 跳转到 ARMProg 入口处开始执行 ARM 指令 (把 R4 设置为 4)

例. 假如 R0 = 0x08000101

⇒ 执行 `BX R0`

跳转到 R0 指向的地址 0x08000100, 由于最低位是 1, 故进入 Thumb 状态.

(bit0 决定进入 ARM/Thumb 状态)

⇒ 执行 `BLX R0`

把返回地址保存到 LR

见 R0.bit0 = 1, 于是 CPSR.T = 1, 进入 Thumb 状态, 跳转到 0x08000100 处执行

例. ARM V5 中的 `LDR PC, [SP], #4` 或 `LDMFD SP!, {R0-R3, PC}` 或 Thumb 中的 `POP {R0-R3, PC}` 这些把值加载到 PC 的指令亦可触发

状态切换, 由于 PC 是程序计数器, 把地址加载到 PC, 本身就是跳转.

若加载到 PC 的地址的 bit0 = 1, 则切换至 Thumb 状态; 反之, 进入 ARM 状态.

1. 先看完整代码结构

.ARM

ADR r0, ThumbProg+1

BX r0

/* 跳到 ThumbProg, 程序切换到 Thumb 状态 */

.THUMB

/* 后面的指令按 Thumb 指令编码 */

ThumbProg:

...

ADR r0, ARMProg

BX r0

/* 跳转到 ARMProg, 程序切换到 ARM 状态 */

.ARM

/* 后面的指令按 ARM 指令编码 */

ARMProg:

MOV r4, #4

2. .ARM 是什么意思？

.ARM 是给汇编器看的，不是 CPU 执行的指令。

意思是：从这里开始，后面的汇编语句要被翻译成 ARM 指令。

ADR r0, ThumbProg+1

BX r0

这两条会被汇编成 ARM 指令。

3. ADR r0, ThumbProg+1 是什么意思？

ADR r0, ThumbProg+1

ADR 的作用是把某个标签地址放进寄存器。

假设：ThumbProg 的真实地址 = 0x00001000

那么：ADR r0, ThumbProg+1

执行后：r0 = 0x00001001

+1 是因为要告诉 BX 要跳到 Thumb 代码，最低位 1 表示目标是 Thumb 状态。

注意实际执行地址还是 0x00001000，最低位 1 只是状态标志。

4. BX r0 怎么切换到 Thumb 状态？

BX r0

CPU 检查 r0 的最低位：r0 = 0x00001001，最低位 bit0 = 1

所以 CPU 做两件事：①切换到 Thumb 状态②跳到地址 0x00001000 执行

也就是：①CPSR.T = 1 ② PC = 0x00001000

于是程序开始执行：

.THUMB

ThumbProg:

...

5. .THUMB 是什么意思？

.THUMB 也是给汇编器看的。

意思是：从这里开始，后面的汇编语句要被翻译成 Thumb 指令。

ThumbProg:

...

ADR r0, ARMProg

BX r0

这些会被汇编成 Thumb 指令。

注意：.THUMB 本身不会让 CPU 切换状态。

BL 自动把返回地址放入 LR.
并跳转到子程序.

真正让 CPU 切换到 Thumb 状态的是前面的：BX r0

6. 从 Thumb 状态切回 ARM 状态

在 Thumb 程序中：

ADR r0, ARMProg

BX r0

假设：ARMProg 的地址 = 0x00002000

那么：ADR r0, ARMProg

执行后：r0 = 0x00002000

这个地址最低位是：bit0 = 0

然后执行：BX r0

CPU 看到：r0 bit0 = 0

于是：

切换到 ARM 状态

跳到 0x00002000

也就是：

CPSR.T = 0

PC = 0x00002000

然后执行：

.ARM

ARMProg:

MOV r4, #4

这条 MOV r4, #4 是 ARM 指令。

7. 最关键的地方：为什么 Thumb 要 +1, ARM 不用？

因为 BX 靠目标地址的最低位判断状态：

Thumb 目标：地址最低位要是 1

ARM 目标：地址最低位要是 0

所以跳到 Thumb 标签时写：

ADR r0, ThumbProg+1

BX r0

跳到 ARM 标签时写：

ADR r0, ARMProg

BX r0

如果写错，比如：

ADR r0, ThumbProg

BX r0

那么 r0 最低位是 0，CPU 会以为目标是 ARM 代码，然后用 ARM 状态去执行 Thumb 指令，程序就会出错。

8. 一句话总结

这段程序的意思是：在 ARM 状态下，把 ThumbProg+1 放入 r0，通过 BX r0 跳到 Thumb 程序并切换到 Thumb 状态；在 Thumb 程序中，把 ARMProg 的地址放入 r0，再通过 BX r0 跳回 ARM 程序并切换回 ARM 状态。

最重要的规则就是：

BX 目标地址 bit0 = 1 → Thumb 状态

BX 目标地址 bit0 = 0 → ARM 状态